Module V – Internet of Things

Course Overview

The Internet of Things (IoT) Laboratory is designed to provide practical exposure to the concepts of IoT systems, including sensing, communication, networking, and cloud integration. The laboratory emphasizes hands-on implementation using embedded systems, enabling students to understand the end-to-end workflow of IoT applications.

Course Objectives

- To understand the architecture and building blocks of IoT systems
- To interface sensors and actuators with embedded platforms
- To implement web-based monitoring and control systems
- To develop communication systems using IoT protocols such as MQTT
- To integrate IoT devices with cloud platforms for data logging and monitoring
- To design and implement complete IoT applications

Hardware and Software Platform

Hardware

- ESP32 Development Board
- Sensors: DHT11/DHT22, LDR
- Actuators: LED, Relay Module
- Breadboard, jumper wires, resistors

Software

- Arduino IDE
- MQTT Client and Broker (e.g., HiveMQ)
- Cloud Platform (e.g., Firebase)

General Instructions

- Verify all hardware connections before powering the system
- Use appropriate GPIO pins as per the microcontroller specifications
- Ensure stable Wi-Fi connectivity during experiments involving networking
- Use serial debugging tools for troubleshooting
- Maintain proper documentation of observations and results

Experiment 1

Title

Implementation of Sensor Data Acquisition System using ESP32

Aim

To acquire temperature and light intensity data using ESP32.

Components Required

- ESP32 Development Board
- DHT11 Sensor
- LDR
- 10kΩ Resistor
- Breadboard and Jumper Wires

Theory

Sensors provide input to IoT systems by converting physical parameters into electrical signals. The ESP32 reads digital data from DHT11 and analog data from LDR using its ADC.

Procedure

1. Connect DHT11 (VCC → 3.3V, GND → GND, DATA → GPIO 4).
2. Connect the LDR in a voltage divider to GPIO 34.
3. Power the ESP32.
4. Open Arduino IDE and install the DHT library.
5. Upload the program.
6. Open Serial Monitor (9600 baud).
7. Observe readings and record values.

Code

```
C/C++
#include "DHT.h"

#define DHTPIN 4
#define DHTTYPE DHT11

#define LED_PIN 2   // Built-in LED (usually GPIO 2)

DHT dht(DHTPIN, DHTTYPE);

int ldrPin = 34;
```

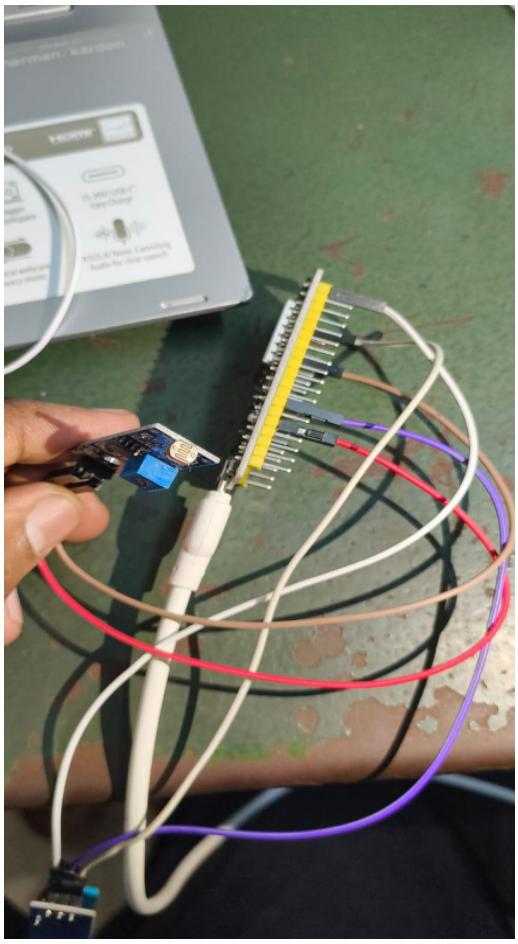
```
void setup() {
  Serial.begin(9600);
  dht.begin();

  pinMode(LED_PIN, OUTPUT);   // Set LED pin as output
}

void loop() {
  float temp = dht.readTemperature();
  int light = analogRead(ldrPin);

  Serial.print("Temp: ");
  Serial.println(temp);

  Serial.print("Light: ");
  Serial.println(light);

  // Simple logic to control built-in LED
  if (light < 2000) {   // Dark condition (adjust threshold as needed)
    digitalWrite(LED_PIN, HIGH);   // LED ON
  } else {
    digitalWrite(LED_PIN, LOW);    // LED OFF
  }

  delay(2000);
}
```

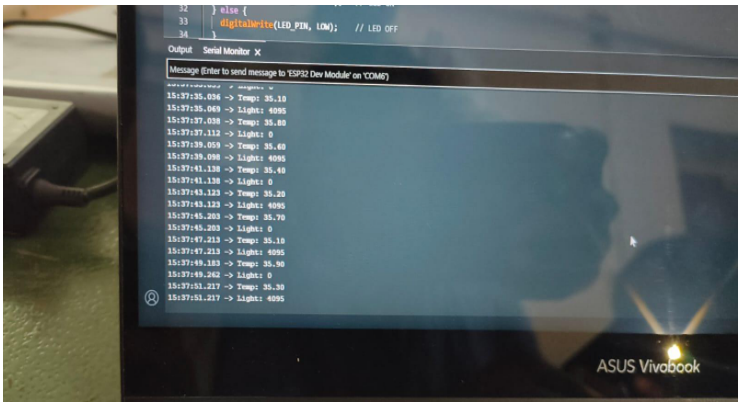
Observation Table

Time (S)	Temperature	Light value	Condition
35.036	35.10	4095	Bright (Led Off)
37.038	35.80	0	Dark (`Led On)
39.059	35.60	4095	Bright (Led Off)
41.138	35.40	0	Dark (`Led On)
43.123	35.20	4095	Bright (Led Off)
45.203	35.70	0	Dark (`Led On)+

47.213	35.10	4095	Bright (Led Off)
49.183	35.90	0	Dark (`Led On)
51.217	35.30	4095	Bright (Led Off)
52.229	35.30	0	Dark (`Led On)

Output Image





Result

Sensor data was successfully acquired using ESP32, and the Built-in LED Turn on.

Experiment 2 - Design of Web-Based LED Control System using ESP32

Aim

To control an LED using a web browser.

Components Required

- ESP32
- LED
- 220Ω Resistor

Theory

ESP32 acts as a web server and responds to HTTP requests from a browser to control hardware.

Procedure

1. Connect the LED to GPIO 2 with a resistor.
2. Enter WiFi credentials in code.
3. Upload program.
4. Open the Serial Monitor to get an IP address.
5. Enter the IP in the browser.
6. Control the LED using a webpage.

Code

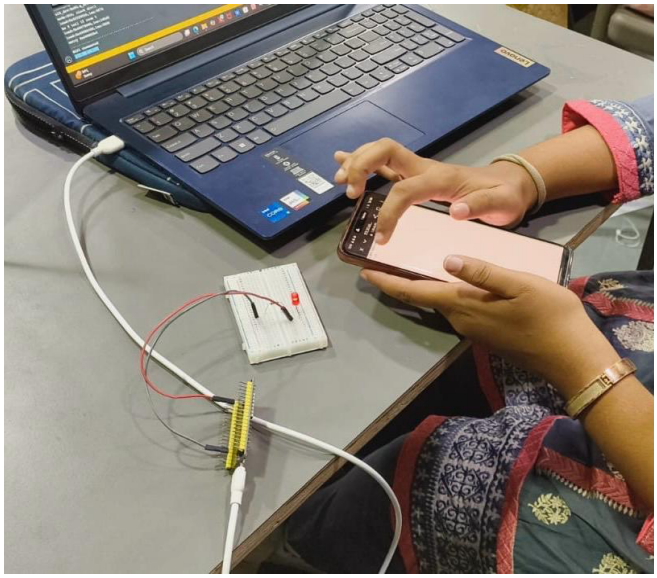
```
C/C++
#include <WiFi.h>

const char* ssid = "YOUR_WIFI";
const char* password = "YOUR_PASSWORD";

WiFiServer server(80);
int ledPin = 2;

void setup() {
 Serial.begin(115200);
 pinMode(ledPin, OUTPUT);

 WiFi.begin(ssid, password);
 while (WiFi.status() != WL_CONNECTED) {
   delay(1000);
 }
```

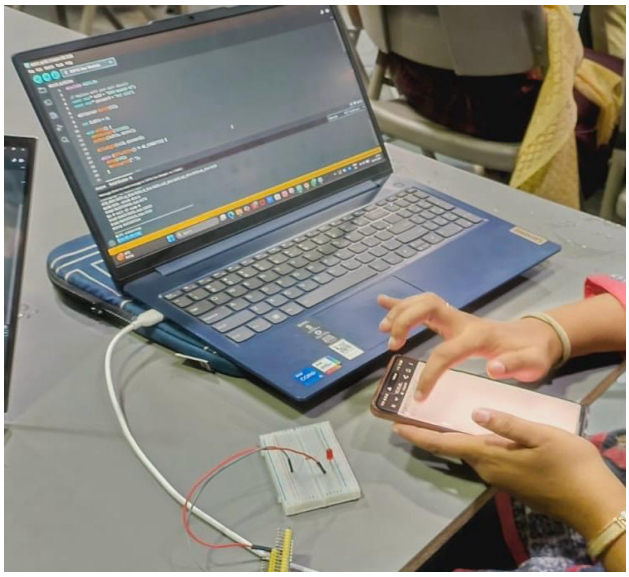
```
 Serial.println(WiFi.localIP());
 server.begin();
}

void loop() {
 WiFiClient client = server.available();
 if (client) {
   String request = client.readStringUntil('\r');

   if (request.indexOf("/ON") != -1) digitalWrite(ledPin, HIGH);
   if (request.indexOf("/OFF") != -1) digitalWrite(ledPin, LOW);

   client.println("HTTP/1.1 200 OK");
   client.println("Content-Type: text/html\n");
   client.println("<a href=\"/ON\">ON</a><br>");
   client.println("<a href=\"/OFF\">OFF</a>");
   client.stop();
 }
}
```

Observation Table

Action	LED Status
ON	ON
OFF	OFF

Output Image





Result

The LED was successfully controlled using a web interface,

Experiment 3

Title

Voice-Controlled Smart Lighting System using ESP32, Adafruit IO, IFTTT, and Relay Module

Aim

To control a 240V AC bulb using voice commands by integrating ESP32 with Adafruit IO cloud and IFTTT platform.

Objectives

- To understand cloud-based IoT communication
- To implement MQTT-based data exchange using Adafruit IO
- To integrate voice control using IFTTT and Google Assistant
- To safely control an AC load using a relay module

Components Required

- ESP32 Development Board
- 1-Channel Relay Module (5V, opto-isolated preferred)
- AC Bulb with holder
- Connecting wires

- Breadboard (for ESP32 side only)

Software / Platforms

- Adafruit IO
- IFTTT
- Google Assistant
- Arduino IDE

Theory

This experiment demonstrates a **cloud-mediated IoT control system**.

Working Principle

1. User gives voice command
2. Google Assistant triggers IFTTT
3. IFTTT updates a feed in Adafruit IO
4. ESP32 subscribes to that feed via MQTT
5. ESP32 receives updated value
6. Relay is triggered
7. AC bulb turns ON/OFF

Circuit Connections

ESP32 to Relay

Relay Pin	ESP32 Connection
VCC	VIN / 5V
GND	GND
IN	GPIO 2

AC Bulb Wiring

Connection	Description
Live Wire	Connected to COM
NO Terminal	Connected to Bulb Live

Neutral	Directly to Bulb

Safety Precautions

- Ensure power is OFF while wiring AC connections
- Do not touch live wires
- Use insulated connectors
- Perform under supervision
- Prefer enclosed relay setup

Procedure

Step 1: Adafruit IO Setup

1. Create an account in Adafruit IO
2. Navigate to “Feeds”
3. Create new feed:

```
None
```

4. `led-control`
5.
6. Note:
    - Username
    - AIO Key

Step 2: ESP32 Programming

1. Open Arduino IDE
2. Install:
    - Adafruit MQTT Library
    - Adafruit IO Library
3. Enter:
    - WiFi credentials
    - Adafruit username
    - AIO key
4. Upload program
5. Open the Serial Monitor to confirm the connection

Step 3: IFTTT Configuration

1. Create a new Applet
2. Select Google Assistant as a trigger
3. Configure phrase:
    - “Turn on light.”
4. Select Adafruit IO as an action
5. Set:
    - Feed: led-control
    - Data: ON

Repeat for OFF command

Step 4: System Testing

1. Power the ESP32
2. Ensure WiFi connection
3. Say:
    - “Turn on light.”
    - “Turn off light.”
4. Observe relay switching
5. Verify bulb ON/OFF
1.

Code

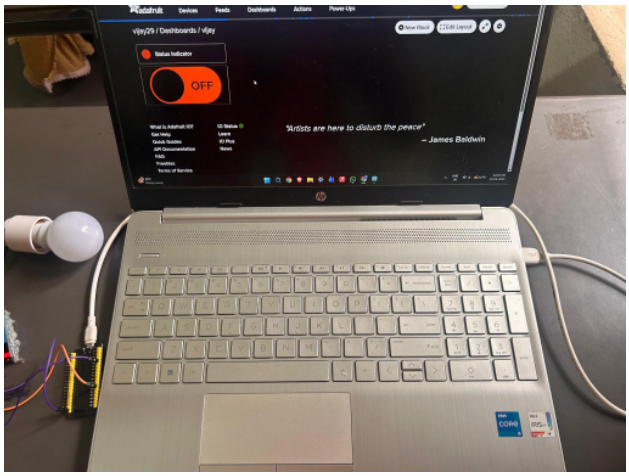
```
C/C++
#include <WiFi.h>
#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"

#define WIFI_SSID "YOUR_WIFI"
#define WIFI_PASS "YOUR_PASSWORD"

#define AIO_SERVER "io.adafruit.com"
#define AIO_SERVERPORT 1883
#define AIO_USERNAME "YOUR_USERNAME"
#define AIO_KEY "YOUR_AIO_KEY"

WiFiClient client;

Adafruit_MQTT_Client mqtt(&client, AIO_SERVER, AIO_SERVERPORT, AIO_USERNAME,
AIO_KEY);

Adafruit_MQTT_Subscribe ledFeed = Adafruit_MQTT_Subscribe(&mqtt, AIO_USERNAME
"/feeds/led-control");
```

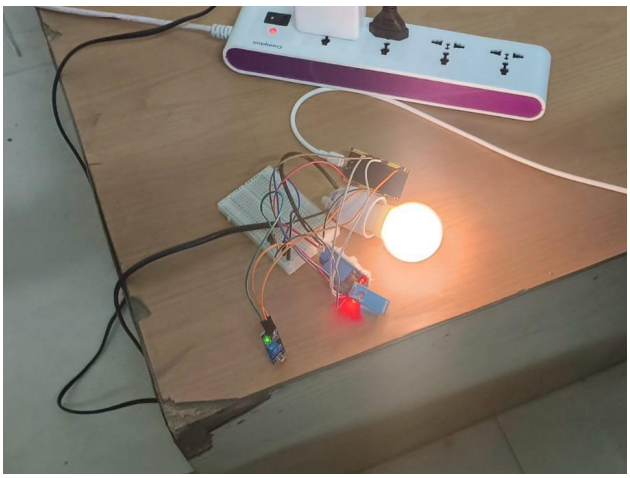
```
int relayPin = 2;

void setup() {
  Serial.begin(115200);
  pinMode(relayPin, OUTPUT);

  WiFi.begin(WIFI_SSID, WIFI_PASS);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
  }

  mqtt.subscribe(&ledFeed);
}

void loop() {
  MQTT_connect();

  Adafruit_MQTT_Subscribe *subscription;
  while ((subscription = mqtt.readSubscription(5000))) {
    if (subscription == &ledFeed) {
      String value = (char *)ledFeed.lastread;

      if (value == "ON") digitalWrite(relayPin, HIGH);
      else digitalWrite(relayPin, LOW);
    }
  }
}

void MQTT_connect() {
  if (mqtt.connected()) return;

  while (mqtt.connect() != 0) {
    delay(5000);
  }
}
```

Observation

Time (Seconds)	Feed Value	Relay State	Bulb Status
3	1	ON	ON
6	0	OFF	OFF

Output Image





Result

The AC bulb was successfully controlled using voice commands through Adafruit IO and IFTTT integration.

Experiment 4

Title

Cloud-Based IoT Monitoring and Control using ESP32 with Firebase

Aim

To store sensor data in the cloud and control an AC bulb using Firebase and web application.

Components Required

- ESP32
- Relay Module (for 240V AC control)
- Bulb
- Sensor DHT11

Theory

Firebase acts as a **real-time database**:

- ESP32 sends sensor data → stored in Firebase
- Web app reads data → displays to user
- User sends command → Firebase updates
- ESP32 reads command → controls relay

Procedure

Step 1: Hardware Setup

1. Connect the relay IN pin to GPIO 2
2. Connect AC bulb through the relay (COM & NO terminals)
3. Ensure proper isolation and safety

Firebase Setup

1. Go to Firebase Console and create a new project
2. Enable **Realtime Database**
3. Select **Test Mode** (for lab use)
4. Copy:
    - Database URL
    - API Key

5. Create structure:

```
None
   /control/relay : "OFF"

   /sensor/value : 0
```

Step 3: ESP32 Programming

1. Connect ESP32 to WiFi
2. Use the Firebase library
3. Read/write values from the database

C/C++

```
#include <WiFi.h>
#include <Firebase_ESP_Client.h>

// WiFi credentials
#define WIFI_SSID "YOUR_WIFI"
#define WIFI_PASSWORD "YOUR_PASSWORD"

// Firebase credentials
#define API_KEY "YOUR_API_KEY"
#define DATABASE_URL "https://YOUR_PROJECT.firebaseio.com/"

// Firebase objects
FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;

int relayPin = 2;
int sensorPin = 34;

void setup() {
  Serial.begin(115200);

  pinMode(relayPin, OUTPUT);
  digitalWrite(relayPin, LOW);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting to WiFi...");
  }

  Serial.println("Connected to WiFi");

  config.api_key = API_KEY;
  config.database_url = DATABASE_URL;

  Firebase.begin(&config, &auth);
  Firebase.reconnectWiFi(true);
}

void loop() {

  // --------- Sensor Upload ---------
  int sensorValue = analogRead(sensorPin);
```

```
  if (Firebase.RTDB.setInt(&fbdo, "/sensor/value", sensorValue)) {
    Serial.println("Sensor updated");
  } else {
    Serial.println(fbdo.errorReason());
  }

  // --------- Read Control ---------
  if (Firebase.RTDB.getString(&fbdo, "/control/relay")) {
    String state = fbdo.stringData();

    if (state == "ON") {
      digitalWrite(relayPin, HIGH);
      Serial.println("Relay ON");
    } else {
      digitalWrite(relayPin, LOW);
      Serial.println("Relay OFF");
    }
  } else {
    Serial.println(fbdo.errorReason());
  }

  delay(3000);
}
```

Step 4: Web App

1. Create simple HTML page
2. Add ON/OFF buttons
3. Update Firebase value on click

HTML

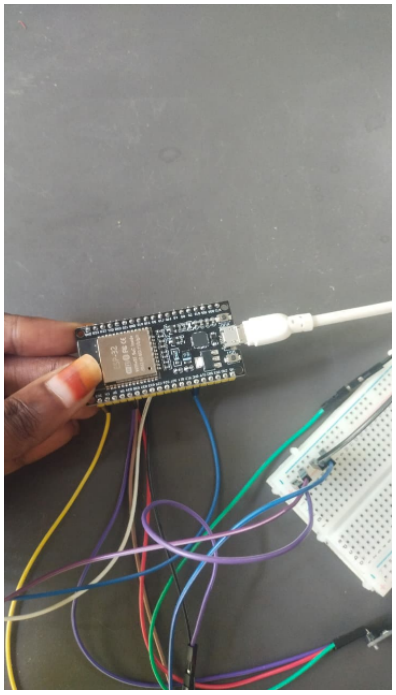
```
#include <WiFi.h>
#include <Firebase_ESP_Client.h>
#include <DHT.h>

// WiFi credentials
#define WIFI_SSID "Muhesh"
#define WIFI_PASSWORD "mk123456"

// Firebase credentials
#define API_KEY "AIzaSyCKW4OEGJHQ02D_jt154_YnFftEksM8ZXQ"
#define DATABASE_URL "https://attamsiot-default-rtdb.firebaseio.com"
```

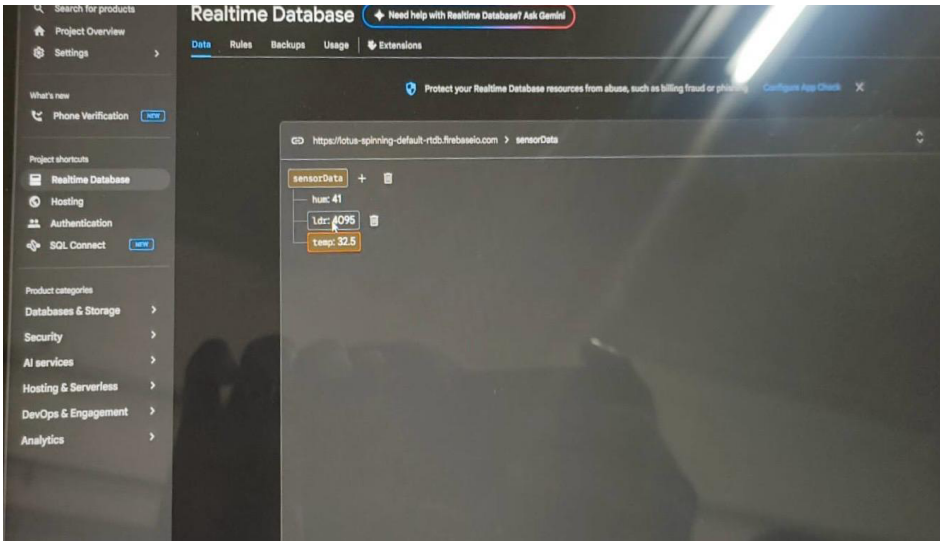
```
// Sensor Pins
#define DHTPIN 5          // DHT11 Data Pin connected to GPIO 5
#define DHTTYPE DHT11
int relayPin = 4;
int ldrPin = 34;

// Objects
DHT dht(DHTPIN, DHTTYPE);
FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;

void setup() {
  Serial.begin(115200);
  dht.begin();

  pinMode(relayPin, OUTPUT);
  digitalWrite(relayPin, LOW);

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting to WiFi...");
  }
  Serial.println("Connected to WiFi");

  config.api_key = API_KEY;
  config.database_url = DATABASE_URL;

  if (Firebase.signUp(&config, &auth, "", "")) {
    Serial.println("Firebase SignUp Success!");
  }

  Firebase.begin(&config, &auth);
  Firebase.reconnectWiFi(true);
}

void loop() {
  // --------- 1. Read Sensors ---------
  float h = dht.readHumidity();
  float t = dht.readTemperature(); // Celsius
  int ldrValue = analogRead(ldrPin);

  // Check if DHT readings are valid
  if (isnan(h) || isnan(t)) {
```

```
    Serial.println("Failed to read from DHT sensor!");
    return;
  }

  // --------- 2. LDR & Relay Logic ---------
  // Adjust 1500 based on your lighting
  int threshold = 1500;
  String relayState;

  if (ldrValue < threshold) {
    digitalWrite(relayPin, HIGH); // Turn Light ON
    relayState = "ON";
  } else {
    digitalWrite(relayPin, LOW);  // Turn Light OFF
    relayState = "OFF";
  }

  // --------- 3. Upload to Firebase ---------
  // We use the "sensorData" path to match your Web UI logic
  Firebase.RTDB.setFloat(&fbdo, "/sensorData/temperature", t);
  Firebase.RTDB.setFloat(&fbdo, "/sensorData/humidity", h);
  Firebase.RTDB.setInt(&fbdo, "/sensorData/ldr", ldrValue);
  Firebase.RTDB.setString(&fbdo, "/control/relay", relayState);

  // Debugging output
  Serial.printf("Temp: %.1f°C | Hum: %.1f%% | LDR: %d | Relay: %s\n", t, h,
ldrValue, relayState.c_str());

  delay(3000); // Wait 3 seconds before next update
}
```

Observation Table

Time	Sensor Value	Relay state	Bulb status
0	32.0	Off	Off
3	32.3	Off	Off
6	32.5	On	On
9	32.7	On	On
12	32.6	Off	Off





Result

- Sensor data was successfully uploaded to Firebase
- The AC bulb was controlled through the web interface
- Real-time communication between ESP32 and the cloud was established